Compiling While++ into Logically Constrained Term Rewriting Systems

Bachelor's thesis presentation

Mikhail Ushakov

Radboud University Nijmegen

Supervisors : Dr C.L.M. Kop
Dr E.G.M. Hubbers

June 27, 2024

Introduction

Motivation

- Imperative programming is the most widespread software development paradigm
- TRSs are effective for static code analysis

Research Question

How can we develop a systematic method of translating imperative programs into the LCTRS?

Term Rewriting Systems

$r1 : max(0, a) \to a$

$r2 : max(a, 0) \to a$

$r3 : max(s(a), s(b)) \to s(max(a, b))$

$r4 : maxl(nil) \to err$

$r5 : maxl(cons(a, l)) \to maxl(pair(a, l))$

$r6 : maxl(pair(a, nil)) \to a$

$r7 : maxl(pair(a, cons(b, l))) \to maxl(pair(max(a, b), l))$

Reduction Example

$$
\begin{aligned}
& \mathit{maxl}(\mathit{cons}(\mathtt{s(s(0))}, \mathit{cons}(\mathtt{s(s(s(0)))}, \mathit{cons}(\mathtt{0}, \mathit{nil})))) \\
\xrightarrow{r5}\ & \mathit{maxl}(\mathit{pair}(\mathtt{s(s(0))}, \mathit{cons}(\mathtt{s(s(s(0)))}, \mathit{cons}(\mathtt{0}, \mathit{nil})))) \\
\xrightarrow{r7}\ & \mathit{maxl}(\mathit{pair}(\mathit{max}(\mathtt{s(s(0))}, \mathtt{s(s(s(0)))}), \mathit{cons}(\mathtt{0}, \mathit{nil})) \\
\xrightarrow{r3}\ & \mathit{maxl}(\mathit{pair}(\mathtt{s}(\mathit{max}(\mathtt{s(0)}, \mathtt{s(s(0))})), \mathit{cons}(\mathtt{0}, \mathit{nil})) \\
\xrightarrow{r3}\ & \mathit{maxl}(\mathit{pair}(\mathtt{s(s}(\mathit{max}(\mathtt{0}, \mathtt{s(0)}))), \mathit{cons}(\mathtt{0}, \mathit{nil})) \\
\xrightarrow{r1}\ & \mathit{maxl}(\mathtt{s(s(s(0)))}, \mathit{cons}(\mathtt{0}, \mathit{nil})) \\
\xrightarrow{r7}\ & \mathit{maxl}(\mathit{pair}(\mathit{max}(\mathtt{s(s(s(0)))}, \mathtt{0})), \mathit{nil}) \\
\xrightarrow{r2}\ & \mathit{maxl}(\mathit{pair}(\mathtt{s(s(s(0)))}, \mathit{nil})) \\
\xrightarrow{r6}\ & \mathtt{s(s(s(0)))}
\end{aligned}
$$

Logically Constrained Term Rewriting Systems

$nil : list$

$cons : [int \times list] \Rightarrow list$

$p : [int \times list] \Rightarrow pair$

$max : [list] \Rightarrow int$

$maxl : [pair] \Rightarrow int$

$r1 : max(nil) \to \texttt{-1}$

$r2 : max(cons(\texttt{a}, l)) \to maxl(pair(\texttt{a}, l))$

$r3 : maxl(pair(\texttt{a}, cons(\texttt{b}, l))) \to maxl(pair(\texttt{a}, l))\ [\texttt{a} \geq \texttt{b}]$

$r4 : maxl(pair(\texttt{a}, cons(\texttt{b}, l))) \to maxl(pair(\texttt{b}, l))\ [\texttt{a} < \texttt{b}]$

$r5 : maxl(pair(\texttt{a}, nil)) \to \texttt{a}$

Reduction Example

$$
\begin{aligned}
& max(cons(2, cons(3, cons(0, nil)))) \\
\xrightarrow{r2}\ & maxl(pair(2, cons(3, cons(0, nil)))) \\
\xrightarrow{r4}\ & maxl(pair(3, cons(0, nil))) && \text{(because } 2 < 3\text{)} \\
\xrightarrow{r3}\ & maxl(pair(3, nil)) && \text{(because } 3 \geq 0\text{)} \\
\xrightarrow{r5}\ & 3
\end{aligned}
$$

While++: Overview

- Originates from While programming language
- Simplistic imperative programming language
- Supports basic concepts of the imperative programming languages
- Well-defined syntax and semantics

While vs. While++

While

```
x := 31;
y := 7;
z := 4;
while y <= x do (
     z := z + 5;
     x := x - y
)
```

While++

```
int x := 0; readInt(x);
int y := 0; readInt(y);
int z := 0; readInt(z);
bool c := y <= x;
while c do (
     z := z + 5;
     x := x - y;
     c := y <= x
);
printString("Answer: ");
printInt(z)
```

While++ Syntax

```
int start := 0;
int end := 100;
int mid := 0;
while not(start = end) do (
    mid := (start + end) / 2;
    bool answer := false;
    printString("Is the number greater than ");
    printInt(mid);
    printString("? (true/false) \n");
    readBool(answer);
    if (answer) then start := mid + 1
    else end := mid
);
printString("The number is ");
printInt(mid);
printString("!")
```

While++ Semantics: Overview

- Natural semantics
- Checking the presence of variables in the program state and verifying their types
- Using expression evaluation function for arithmetic/boolean/string expressions
- Defined semantics for I/O lists
- Managing the variables scope in case of **if** and **while** statements

While++ Semantics: Example Rules

Rule [declare$_{int}$]

$$\langle int\ var\ :=\ expr, s\rangle \to s[var \mapsto \mathcal{E}val_{int}[\![expr]\!]s] \text{ if } \varphi$$

$$\varphi = \forall x' \in Dom(s) : x' \neq var \ \wedge\ var \in \mathbf{Var}_{int} \ \wedge\ \mathcal{E}val_{type}[\![expr]\!]s \downarrow$$

Rule [print$_{string}$]

$$\langle \texttt{printString}(expr), s\rangle \to s[\texttt{Out} \mapsto \mathbf{consS}(\mathcal{E}val_{string}[\![expr]\!]s, s\ \texttt{Out})] \text{ if } \varphi$$

$$\varphi = var \in Dom(s) \ \wedge \texttt{In} \in Dom(s) \wedge$$
$$\mathbf{headS}(s\ \texttt{In}) \downarrow \ \wedge\ \mathbf{tail}(s\ \texttt{In}) \downarrow$$

Rule [while$^{\top}_{ns}$]

$$\frac{\langle S, s\rangle \to s' \quad \langle \texttt{while}\ b\ \texttt{do}\ S, LeaveScope(s, s')\rangle \to s''}{\langle \texttt{while}\ b\ \texttt{do}\ S, s\rangle \to LeaveScope(s, s'')} \text{ if } \mathcal{B}[\![b]\!]s$$

Translation Functions

- Translating variables from semantics representation into LCTRS variables

$$\mathcal{VT} : \mathbf{Var} \cup \mathbf{Var}_{list} \cup \overrightarrow{\mathbf{Var} \cup \mathbf{Var}_{list}} \to \mathcal{V} \cup \vec{\mathcal{V}}$$

- Translating arithmetic/boolean/string expressions into the LCTRS terms

$$\mathcal{AT} : \mathbf{Aexp} \to \mathcal{T}erms(\Sigma_{theory} \cup \mathcal{V})$$
$$\mathcal{BT} : \mathbf{Bexp} \to \mathcal{T}erms(\Sigma_{theory} \cup \mathcal{V})$$
$$\mathcal{S}tr\mathcal{T} : \mathbf{Strexp} \to \mathcal{T}erms(\Sigma_{theory} \cup \mathcal{V})$$

- Translating While++ programs into the LCTRS

$$\mathcal{T}(S, \vec{x}, id) = ((\Sigma_S, \mathcal{R}_S), \vec{x'}, id')$$

- Using the output of the translation function $\mathcal{T}$, we can construct an LCTRS

$$\begin{pmatrix} \Sigma_{theory} \cup \Sigma_{list} \cup \Sigma_S, \\ \mathcal{R}_{list} \cup \mathcal{R}_S, \\ \mathcal{I}, \mathcal{J} \end{pmatrix}$$

Translation Functions: Examples

Translation rule for "print" statement for arithmetic expressions

$$\frac{(|\vec{x}| = |\vec{w}|) \wedge \exists p \in [0, |\vec{x}| - 1] : \begin{pmatrix} (x_p = \texttt{Out}) \wedge (w_p = \mathit{consl}(\mathcal{AT}(\mathit{expr}), \mathcal{VT}(\texttt{Out}))) \wedge \\ (\forall i \in \{0, |\vec{x}| - 1\} : (i \neq p) \implies (w_i = \mathcal{AT}(x_i))) \end{pmatrix}}{[\![\texttt{printInt}(\mathit{expr})]\!]_{\vec{x},id} = \left(\begin{pmatrix} \{\mathit{stm}_{id+1} : \mathit{signature}(\mathcal{VT}(\vec{x}), \mathit{list}), \\ \mathit{stm}_{id+2} : \mathit{signature}(\mathcal{VT}(\vec{x}), \mathit{list})\}, \\ \{\mathit{stm}_{id+1}[\mathcal{VT}(\vec{x})] \rightarrow \mathit{stm}_{id+2}[\vec{w}]\ [\mathit{true}]\} \end{pmatrix}, \vec{x}, id + 2 \right)}$$

Translation rule for "if" statement

$$\frac{[\![S_1]\!]_{\vec{x},id+1} = \left(\begin{pmatrix} \Sigma_{S_1}, \\ \mathcal{R}_{S_1} \end{pmatrix}, \vec{x} \cdot \vec{y}, id' \right) \qquad [\![S_2]\!]_{\vec{x},id'} = \left(\begin{pmatrix} \Sigma_{S_2}, \\ \mathcal{R}_{S_2} \end{pmatrix}, \vec{x} \cdot \vec{z}, id'' \right)}{[\![\texttt{if } b \texttt{ then } S_1 \texttt{ else } S_2]\!]_{\vec{x},id} = \left(\begin{pmatrix} \Sigma_{S_1} \cup \Sigma_{S_2} \cup \\ \{\mathit{stm}_{id+1} : \mathit{signature}(\mathcal{VT}(\vec{x}), \mathit{list}), \\ \mathit{stm}_{id''+1} : \mathit{signature}(\mathcal{VT}(\vec{x}), \mathit{list})\}, \\ \{\mathit{stm}_{id+1}[\mathcal{VT}(\vec{x})] \rightarrow \mathit{stm}_{id+2}[\mathcal{VT}(\vec{x})]\ [\mathcal{BT}(b)]\} \cup \mathcal{R}_{S_1} \cup \\ \{\mathit{stm}_{id+1}[\mathcal{VT}(\vec{x})] \rightarrow \mathit{stm}_{id'+1}[\mathcal{VT}(\vec{x})]\ [\mathit{not}(\mathcal{BT}(b))]\} \cup \mathcal{R}_{S_2} \cup \\ \{\mathit{stm}_{id'}[\mathcal{VT}(\vec{x} \cdot \vec{y})] \rightarrow \mathit{stm}_{id''+1}[\mathcal{VT}(\vec{x})]\ [\mathit{true}], \\ \mathit{stm}_{id''}[\mathcal{VT}(\vec{x} \cdot \vec{z})] \rightarrow \mathit{stm}_{id''+1}[\mathcal{VT}(\vec{x})]\ [\mathit{true}]\} \end{pmatrix}, \vec{x}, id'' + 1 \right)}$$

Translating While++ Programs into LCTRS

$$\mathcal{T}(P_1, [\texttt{In} \in \mathbf{Var}_{list}, \texttt{Out} \in \mathbf{Var}_{list}], 0)$$
$$= (\Sigma_{P_1}, \mathcal{R}_{P_1}, [\texttt{In} \in \mathbf{Var}_{list}, \texttt{Out} \in \mathbf{Var}_{list}, \texttt{x} \in \mathbf{Var}_{int}], 6)$$

Program P_1

```
int x := 0;
while x < 10 do (
    x := x + 1
)
```

Signature Σ_{P_1}

$stm_1 : [list \times list] \Longrightarrow list$
$stm_2 : [list \times list \times int] \Longrightarrow list$
$stm_3 : [list \times list \times int] \Longrightarrow list$
$stm_4 : [list \times list \times int] \Longrightarrow list$
$stm_5 : [list \times list \times int] \Longrightarrow list$
$stm_6 : [list \times list \times int] \Longrightarrow list$

Set of LCTRS rules $\mathcal{R}_{P_1}$

$stm_1(in, out) \to stm_2(in, out, 0)\ [true]$
$stm_2(in, out, x) \to stm_3(in, out, x)\ [true]$
$stm_3(in, out, x) \to stm_4(in, out, x)\ [<(x, 10)]$
$stm_4(in, out, x) \to stm_5(in, out, +(x, 1))\ [true]$
$stm_5(in, out, x) \to stm_3(in, out, x)\ [true]$
$stm_3(in, out, x) \to stm_6(in, out, x)\ [not(<(x, 10))]$

We can add one more rule:

$stm_6(in, out, x) \to out\ [true]$

Correctness Theorem

$$
\begin{aligned}
\mathit{CorrectnessTheorem}(\mathcal{T}) \;:= \forall & S \in \mathcal{S}tm, s \in \mathcal{S}tate, \vec{x} \in \overrightarrow{\mathbf{Var}}, \\
& id \in \mathbb{N}, \vec{v} \in \overrightarrow{\mathcal{T}erms(\Sigma_{theory} \cup \Sigma_{list})}, \\
& (\Sigma_S, \mathcal{R}_S), \vec{x'} \in \overrightarrow{\mathbf{Var}}, id' \in \mathbb{N} : \\
& \quad \mathit{Requirements}(S, s, \vec{x}, id, (\Sigma_S, \mathcal{R}_S), \vec{v}, \vec{x'}, id') \implies \\
& \quad\quad \mathit{Soundness}(S, s, \vec{x'}, \vec{v}, \mathcal{R}_S, id + 1, id') \wedge \\
& \quad\quad \mathit{Completeness}(S, s, \Sigma_S, \mathcal{R}_S, id + 1, \vec{v})
\end{aligned}
$$

We want to verify whether, for all While++ programs, we can generate an equivalent LCTRS using the translation function $\mathcal{T}$.

Correctness Theorem: Requirements

$$
\begin{aligned}
\mathit{Requirements}(S, s, \vec{x}, id, (\Sigma_S, \mathcal{R}_S), \vec{v}, \vec{x'}, id') \;:=\; & \mathcal{T}(S, \vec{x}, id) = ((\Sigma_S, \mathcal{R}_S), \vec{x'}, id') \wedge \\
& |\vec{x}| = |\mathit{Dom}(s)| = |\vec{v}| \wedge \\
& (\forall x_i \in \vec{x} : x_i \in \mathit{Dom}(s)) \wedge \\
& (\forall i \in \{0, |\vec{v}| - 1\} : v_i \in \mathit{NF}(\rightarrow_{\mathcal{R}_{list}})) \wedge \\
& (\forall i \in \{0, |\vec{x}| - 1\} : s\ x_i = [\![v_i]\!]_{combi})
\end{aligned}
$$

We verify whether the translation function is defined for S, $\vec{x}$, id. And also, we are checking whether the term $stm_{id+1}[\vec{v}]$ effectively represents the initial state in the derivation tree: s.

Correctness Theorem: Soundness

$$
\begin{aligned}
Soundness(S, s, \vec{x'}, \vec{v}, \mathcal{R}_S, id, id') \;:=\; & \forall s' \in \mathcal{S}tate : \\
& \langle S, s\rangle \to s' \implies \\
& \quad (\forall x'_i \in \vec{x'} : x'_i \in Dom(s')) \wedge \\
& \quad \exists v' \in \overrightarrow{\mathcal{T}erms(\Sigma_{theory} \cup \Sigma_{list})} : \\
& \quad \begin{pmatrix} stm_{id+1}[\vec{v}] \to^*_{\mathcal{R}_{list} \cup \mathcal{R}_S} stm_{id'}[\vec{v'}] \wedge \\ stm_{id'}[\vec{v'}] \in NF(\to_{\mathcal{R}_{list} \cup \mathcal{R}_S}) \wedge \\ |\vec{x'}| = |Dom(s')| = |\vec{v'}| \wedge \\ (\forall i \in \{0, |\vec{x'}| - 1\} : s'\ x'_i = [\![v'_i]\!]_{combi}) \end{pmatrix}
\end{aligned}
$$

We are checking if the exists a derivation tree with the conclusion $\langle S, s\rangle \to s'$ then the term $stm_{id+1}[\vec{v}]$ should reduce to the normal form $stm_{id'}[\vec{v'}]$. This normal form should effectively represent the final state in the derivation tree: s'.

Correctness Theorem: Completeness

Not proven within this thesis project!

$$
\begin{aligned}
Completeness(S, s, \Sigma_S, \mathcal{R}_S, id, id', \vec{v}, \vec{v'}) \; := \neg\exists s' \in \mathcal{S}tate : \\
\langle S, s\rangle \to s' \implies \\
\neg\exists stm_{id'}[\vec{v'}] \in \mathcal{T}erms(\Sigma_{theory} \cup \Sigma_{list} \cup \Sigma_S) : \\
\begin{pmatrix} stm_{id+1}[\vec{v}] \to^*_{\mathcal{R}_{list} \cup \mathcal{R}_S} stm_{id'}[\vec{v'}] \wedge \\ stm_{id'}[\vec{v'}] \in NF(\to_{\mathcal{R}_{list} \cup \mathcal{R}_S}) \end{pmatrix}
\end{aligned}
$$

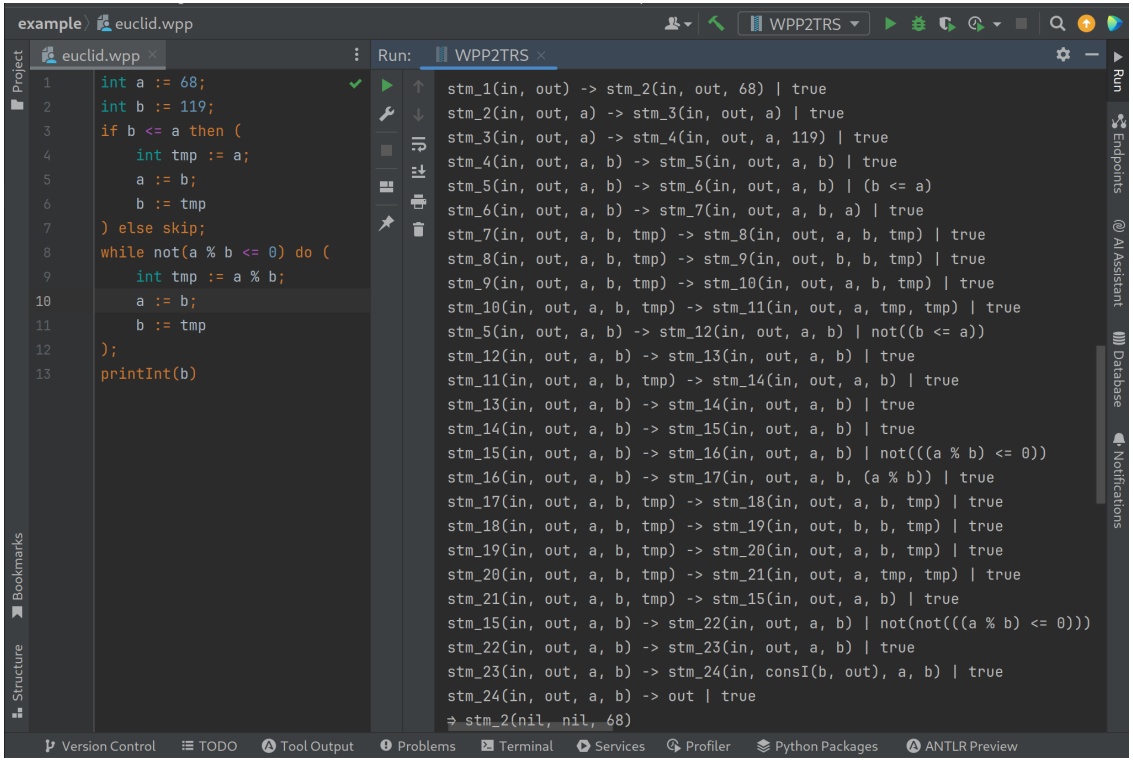
We are verifying if there doesn't exist a derivation tree with the conclusion $\langle S, s\rangle \to s'$, then there should not exist a normal form $stm_{id'}[\vec{v'}]$, to which $stm_{id+1}[\vec{v}]$ can be reduced to.

Soundness Proof: Overview

- Structural induction on the shape of the derivation tree
- First, we assume that $\mathit{Requirements}(S, s, \vec{x}, id, (\Sigma_S, \mathcal{R}_S), \vec{v}, \vec{x'}, id')$ holds
- Then, for all While++ natural semantics rules, we assume that this rule was applied last
- We verify whether $\mathit{Soundness}(S, s, \vec{x'}, \vec{v}, \mathcal{R}_S, id, id')$ holds in this case
  - For axiom rules such as $[\mathrm{skip}_{\mathrm{ns}}]$ or $[\mathrm{declare}_{\mathrm{int}}]$ the verification of conditions is straightforward
  - For complex rules such as $[\mathrm{comp}_{\mathrm{ns}}]$ or $[\mathrm{while}^{\top}_{\mathrm{ns}}]$ we need to use an induction hypothesis(es)

WPP2TRS

- Java library for translating While++ into LCTRS
- Parser for While++ is implemented using ANTLR
- Output LCTRS is compatible with `Cora`



Conclusion

What did we achieve?

- Constructed a simplistic imperative programming language
- Developed the translation method
- Proved the soundness of the translation
- Implemented the Java library

Future Work

What's next?

- Prove the completeness of the translation
- Extend the While++ further:
  - Add complex data structures: *arrays*, *lists*, ...
  - Add more complex language constructions: *functions*, *lambdas*, *exceptions*, ...
- Develop and prove the translation methods for these new language constructions